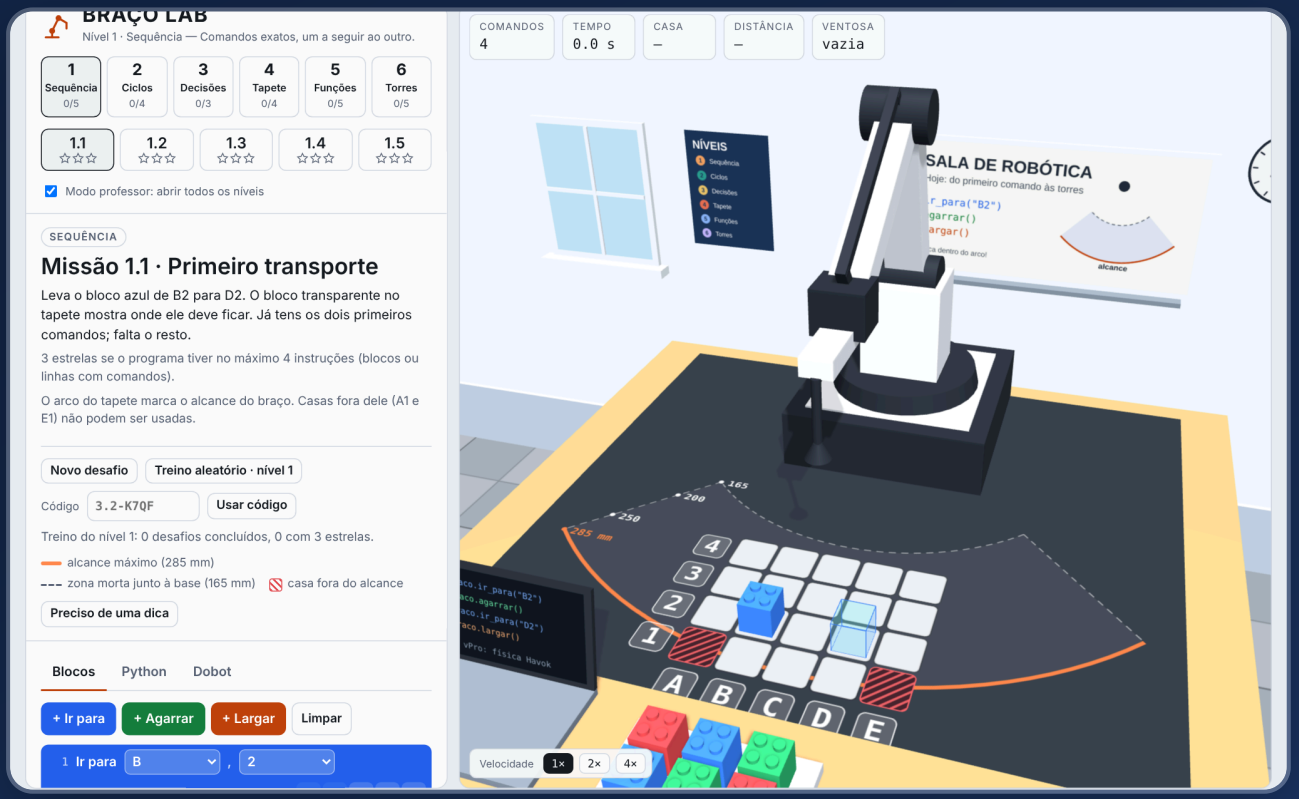
MANUAL TÉCNICO E GUIA DO ALUNO

Braço Lab vPro 2

Simulador 3D do braço robótico Dobot Magician: programar com blocos e Python, resolver missões e passar ao robô real.

26 MISSÕES · 6 NÍVEIS · BLOCOS + PYTHON



Versão 2.0 · Babylon.js + Havok

Edição 2026

Índice

1 **Introdução** 2
O que é o Braço Lab e para que serve

2 **Começar** 4
Como abrir o simulador e o que é preciso

3 **O ecrã** 5
Visita guiada pelo painel e pela sala 3D

4 **Como jogar** 7
Missões, níveis, estrelas e dicas

5 **Programar com blocos** 9
Instruções coloridas que encaixam umas nas outras

6 **Programar em Python** 11
O mesmo robô, agora com texto

7 **A mesa e as medidas** 13
Geometria, alcance e coordenadas do robô

8 **Tempo e eficiência** 15
Quanto custa cada movimento

9 **Níveis e missões** 16
O mapa completo das 26 missões

10 **O tapete rolante** 18
Receber blocos que chegam sozinhos

11 **Torres e colisões** 20
Empilhar em segurança (nível 6)

12 **Desafios aleatórios** 22
Treinar sem repetir sempre o mesmo exercício

13 **Do simulador ao Dobot real** 23
Gerar o script e calibrar a mesa

14 **Depurar e corrigir** 25
Quando o programa não faz o que querias

15 **Guia do professor** 26
Preparar e conduzir uma aula

16 **Resolução de problemas** 27
Quando é o simulador que não responde

17 **Ficha de consulta rápida** 28
Tudo o que precisas numa página, e glossário

COMO USAR ESTE MANUAL

Os capítulos 1 a 8 explicam o simulador e as medidas; 9 a 12 o percurso de aprendizagem; 13 a 16 o braço real, o professor e os problemas; o 17 é uma ficha para ter à mão. As caixas escuras resumem o objetivo, as verdes são dicas, as amarelas avisos e as roxas propõem experiências.

1 Introdução

O que é o Braço Lab e para que serve

NESTE CAPÍTULO

- Perceber a ideia do simulador e o percurso do aluno
- Conhecer as três versões e escolher a certa para a aula

O **Braço Lab** é um simulador 3D de um braço robótico **Dobot Magician**, com ventosa, sensor de cor e tapete rolante. O aluno escreve um programa, vê o braço executá-lo numa sala virtual e, quando está satisfeito, leva exatamente o mesmo programa para o braço real. Treina-se sem risco, sem desgaste do equipamento e sem fila para usar o robô.

Funciona no navegador, em computador, tablet ou telemóvel, sem instalar nada. Cada missão tem um objetivo claro, o simulador valida a solução e atribui até três estrelas pela eficiência do programa.

6	26	3	∞
NÍVEIS	MISSÕES	FORMAS DE PROGRAMAR	DESAFIOS ALEATÓRIOS

O percurso do aluno

Passo	O que acontece
1	**Ler a missão.** O enunciado explica o objetivo; os blocos transparentes na mesa mostram onde cada bloco deve ficar.
2	**Programar** com blocos coloridos ou em Python. Os dois separadores mostram sempre o mesmo programa.
3	**Correr** e observar o braço. Se falhar, o simulador diz o que correu mal e em que linha; o depurador mostra as variáveis passo a passo.
4	**Ganhar estrelas** com um programa correto e curto. Os cenários escondidos garantem que a solução funciona em geral, e não só no exemplo.
5	**Treinar** com desafios aleatórios do mesmo tipo, para consolidar sem repetir sempre o mesmo exercício.
6	**Passar ao braço real** no separador Dobot: o simulador gera o script para o Dobot Magician.

Versões disponíveis

Este manual descreve a **vPro 2**. Tudo o que não está identificado como novo existe também nas versões anteriores.

Versão	Motor 3D	O que traz	Quando usar
Braço Lab	Three.js	21 missões em 5 níveis, depurador, cenários escondidos, calibração guiada	Níveis 1 a 5; é a mais leve
Braço Lab vPro	Babylon.js + física Havok	Empilhamento até 3 blocos, comando `braco.altura(n)`, colisões com queda de torres, nível 6 (26 missões)	Para trabalhar torres e colisões

Versão	Motor 3D	O que traz	Quando usar
Braço Lab vPro 2	Babylon.js + física Havok	Tudo da vPro, mais desafios aleatórios, códigos para a turma e treino por nível	Para treinar sem repetir exercícios

2 Começar

Como abrir o simulador e o que é preciso

NESTE CAPÍTULO

- Abrir a versão online ou a versão offline
- Saber o que o computador precisa e onde fica guardado o progresso

Duas formas de abrir

ONLINE

Abre a página publicada (o endereço dado pelo professor). Na primeira abertura a página descarrega o Babylon.js e a física Havok a partir de uma rede de distribuição, por isso precisa de internet.

OFFLINE

Abre o ficheiro `braco-lab-vpro2-offline.html` com duplo clique. Tem tudo incluído (cerca de 12 MB) e funciona sem rede, por exemplo numa sala sem Wi-Fi.

Requisitos

Item	Requisito
Navegador	Chrome, Edge, Firefox ou Safari recentes, com WebGL ativo
Equipamento	Qualquer computador com placa gráfica integrada; tablet e telemóvel funcionam (a sala fica por cima e o painel por baixo)
Rede	Só para a versão online; a offline não precisa
Instalação	Nenhuma
Dobot real	Opcional, só para o capítulo 13

Física e alternativa

A queda dos blocos e o desabamento das torres usam o motor físico **Havok** (WebAssembly). Se o navegador ou a escola bloquear o WebAssembly, a página continua a funcionar com uma animação de queda mais simples: os blocos ficam sempre no sítio certo e o programa continua a detetar colisões. O ficheiro offline corre sempre com a física completa.

Onde fica o progresso

As estrelas, o modo professor e as contagens do treino ficam guardados **no navegador do aluno**. Mudar de navegador ou de computador, ou limpar os dados do site, apaga-os. O texto dos programas **não** é guardado entre sessões: copia-o se o quiseres conservar.

ATENÇÃO

Em computadores partilhados, o progresso de um aluno aparece ao seguinte se usarem o mesmo navegador e o mesmo utilizador do sistema. Para começar do zero, usa uma janela privada ou limpa os dados do site.

3 O ecrã

Visita guiada pelo painel e pela sala 3D

NESTE CAPÍTULO

- Identificar cada zona do ecrã
- Mexer a câmara e mudar a velocidade

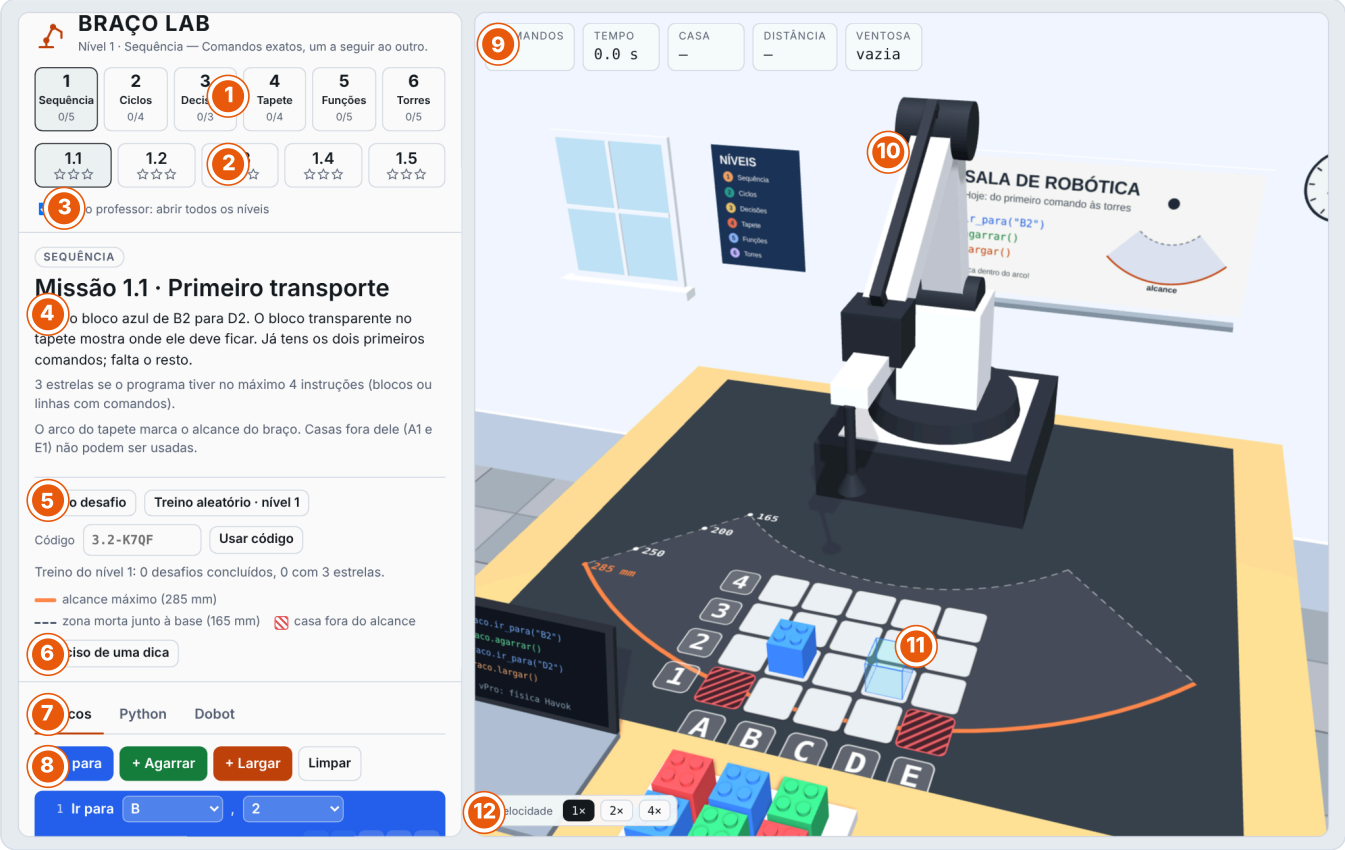


Figura 3.1 · O ecrã do simulador com as zonas numeradas (a tabela seguinte explica cada uma).

N.º	Zona	Para que serve
1	Níveis	Seis botões. Mostram as missões concluídas (por exemplo 3/5). Cada nível só abre quando o anterior está concluído.
2	Missões do nível	Um botão por missão (1.1, 1.2…), com as estrelas ganhas.
3	Modo professor	Abre todos os níveis e liberta os desafios aleatórios sem exigir a missão guiada.
4	Enunciado	Conceito, título, texto, limite de instruções para 3 estrelas e legenda do tapete.
5	Desafios aleatórios	Novo desafio, Treino aleatório e campo Código (capítulo 12).
6	Dicas	«Preciso de uma dica» mostra uma dica de cada vez. Há duas por missão.
7	Separadores	**Blocos**, **Python** e **Dobot**.
8	Paleta de blocos	Os blocos disponíveis na missão. Em missões só de Python não aparece.
9	Indicadores (HUD)	Comandos executados, tempo simulado, casa atual, distância à base (mm) e conteúdo da ventosa.

N.º	Zona	Para que serve
10	O braço	Dobot Magician virtual, com base, ombro, cotovelo e ventosa.
11	A mesa	Casas A a E e 1 a 4, arco de alcance e blocos-alvo transparentes.
12	Velocidade	1×, 2× e 4×. Muda só a rapidez da animação, não o tempo simulado nem as estrelas.

Os botões de execução

Botão	Função
▶ **Correr**	Executa o programa em todos os cenários da missão e valida o resultado.
■ **Parar**	Interrompe a execução.
⏸ **Pausa**	Pára a meio; mostra as variáveis do programa.
⏭ **Passo**	Executa um comando de cada vez.
↺ **Repor**	Volta a pôr a mesa como no início do cenário.

A câmara

- **Rodar:** arrasta com o rato ou com o dedo.
- **Aproximar e afastar:** roda do rato ou gesto de pinça.
- **Limites:** a câmara nunca sai da sala. Roda cerca de 57° para cada lado, inclina-se entre quase de topo e quase de frente, e o zoom vai de 55% a 115% da distância inicial.

A página segue o tema do sistema (claro ou escuro) e a sala adapta as cores e a luz.

4 Como jogar

Missões, níveis, estrelas e dicas

NESTE CAPÍTULO

- Concluir uma missão do princípio ao fim
- Perceber como se ganham estrelas e se desbloqueiam níveis

A tua primeira missão em cinco passos

Missão 1.1 · *Primeiro transporte*: levar o bloco azul de B2 para D2.

1. **Lê o enunciado.** O bloco azul está em B2 e o bloco transparente em D2 mostra o destino.
2. **Escolhe o separador Blocos** e carrega em **+ Ir para**. Escolhe a coluna **B** e a linha **2**.
3. Acrescenta **+ Agarrar**, depois outro **+ Ir para** (coluna **D**, linha **2**) e por fim **+ Largar**.
4. Carrega em **▶ Correr** e vê o braço ir a B2, agarrar, deslocar-se a D2 e largar.
5. Lê o resultado: *Missão concluída*, com as estrelas ganhas.

```python
# o mesmo programa em Python
braco.ir_para("B2")
braco.agarrar()
braco.ir_para("D2")
braco.largar()
```

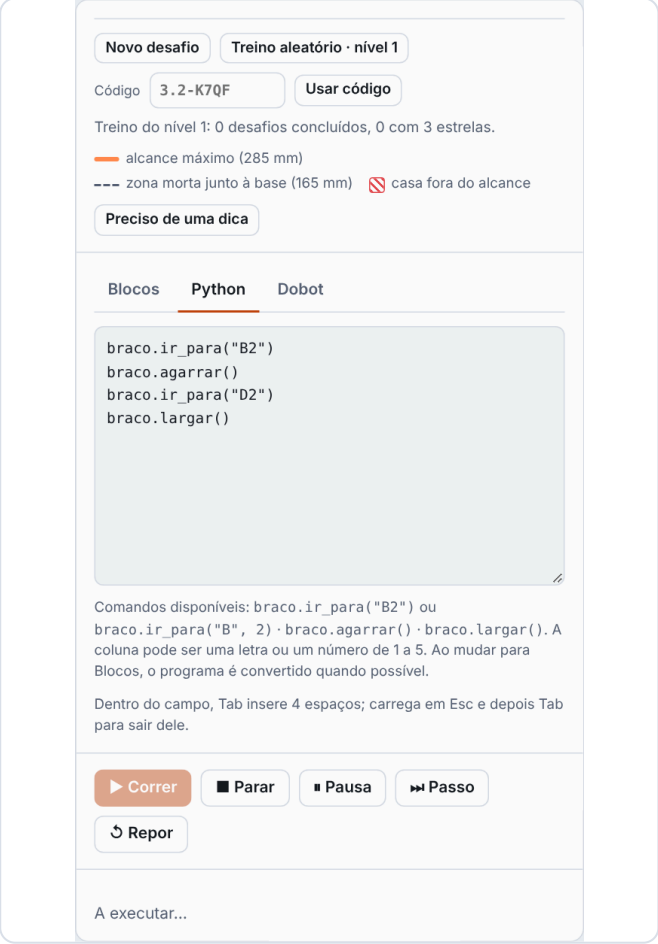


Figura 4.1 · Resultado de uma missão concluída.

Estrelas

As estrelas medem a **eficiência**: quantas instruções o programa tem. Cada missão tem um limite («par»); um bloco ou uma linha com comando conta como uma instrução.

Estrelas	Condição
★★★	Instruções ≤ par
★★☆	Instruções ≤ 1,5 × par (arredondado para cima)
★☆☆	Qualquer solução correta, mais longa do que isso

Só a primeira conclusão conta para o progresso; voltar a correr não retira estrelas e uma solução melhor aumenta-as.

> **DICA**
>
> Se uma missão tem par 5 e a tua solução tem 12 blocos iguais, falta um **ciclo**. As estrelas só desafiam a pensar de forma mais inteligente, nunca a escrever menos por escrever.

Níveis e desbloqueio

O nível 1 está sempre aberto. Cada nível seguinte abre quando todas as missões do anterior têm pelo menos uma estrela. O **modo professor** (caixa por baixo dos botões de missão) abre todos os níveis.

Dicas e cenários

- **Preciso de uma dica** mostra a primeira dica; carregar de novo mostra a seguinte. Evita ir logo à solução: a primeira dica orienta, a segunda quase diz o caminho.
- **Cenários**: algumas missões têm vários cenários (botões 1, 2, 3). O programa tem de funcionar em todos. Clica num para o ver na mesa.

> **EXPERIMENTA**
>
> 1. Faz a missão 1.1 e anota quantas instruções gastaste.
> 2. Altera o destino para B4. O que muda no programa?
> 3. Corre com velocidade 4× e compara o **Tempo** mostrado nos indicadores com o de 1×. O que reparas?

5 Programar com blocos

Instruções coloridas que encaixam umas nas outras

NESTE CAPÍTULO

- Montar, editar e mover blocos
- Perceber como o programa em blocos se transforma em Python

No separador **Blocos**, a paleta mostra os blocos disponíveis na missão. Carregar num bloco acrescenta-o ao fim do programa; os campos de cada bloco são menus que também aceitam texto livre.

Os blocos

Bloco	O que faz	Nível
Ir para	Desloca a ventosa para uma casa. Campos: **coluna** (A a E, T ou número 1 a 5) e **linha** (1 a 4).	1
Agarrar	Desce, liga a sucção e sobe com o bloco de cima da casa.	1
Largar	Desce, desliga a sucção e pousa o bloco (por cima de outros, se houver).	1
Repetir	Ciclo com contador: *repetir i de 1 a 4*. O corpo fica dentro do bloco.	2
Se cor / Senão	Decisão com o sensor de cor: *se cor é vermelho... senão...*	3
Avançar tapete	Faz chegar o próximo bloco ao ponto T do tapete rolante.	4
Altura	Define o nível da ventosa em viagem (1 a 5). Serve para passar por cima de torres.	6

Editar o programa

- ↑ ↓ sobem e descem um bloco.
- → ← aumentam ou diminuem a indentação, isto é, põem um bloco dentro de um *Repetir* ou de um *Se* (ou tiram-no).
- × apaga o bloco.
- **Limpar** apaga o programa todo.

Nos campos podes escolher da lista ou escrever: letras de coluna, números, a variável do ciclo (`i`, `j`) ou contas simples como `i - 1`.



Figura 5.1 · Paleta de blocos e primeiros blocos do programa.

Blocos e Python andam juntos

Ao acrescentar um bloco, o código Python por baixo atualiza-se. Ao passar de Python para Blocos, o programa é convertido **quando for possível**. Se usares algo que não tem bloco (por exemplo `while`, funções ou variáveis), o simulador avisa e mantém a versão anterior dos blocos.

> **ATENÇÃO**
> Algumas missões (as de funções e a «Até esvaziar») são **só em Python**: o separador Blocos fica inativo e o ecrã diz «Esta missão é só em Python».

> **EXPERIMENTA**
> Faz a missão 2.1 em Blocos. Depois abre o separador Python e identifica a linha `for i in range(1, 5):`. Que bloco lhe corresponde? Onde começa e onde acaba o corpo do ciclo?

6 Programar em Python

O mesmo robô, agora com texto

NESTE CAPÍTULO

- Escrever programas com o objeto `braco`
- Conhecer a parte de Python que o simulador entende

No separador **Python** escreves o programa como texto. O simulador inclui um interpretador próprio, seguro e pequeno: aceita a parte de Python que é útil para o robô e dá mensagens de erro em português, com o número da linha.

Os comandos do braço

Comando	Efeito
`braco.ir_para("B2")`	Move a ventosa para a casa. Aceita também `ir_para("T")` (ponto de recolha do tapete) e `ir_para("B", 2)` ou `ir_para(2, 2)` (coluna e linha).
`braco.agarrar()`	Desce, liga a sucção e sobe com o bloco de cima da casa onde está. Em T, apanha o bloco que chegou ao tapete.
`braco.largar()`	Desce, desliga a sucção e pousa o bloco no topo da casa (não se pode largar no tapete).
`braco.altura(n)`	Define a altura de viagem, de 1 a 5 (nível 6).
`tapete.avancar()`	Faz chegar o próximo bloco ao ponto T (nível 4). Só avança se T estiver livre.
`tapete.tem_bloco()`	Verdadeiro se há um bloco em T à espera de ser recolhido.
`sensor.cor()`	Devolve a cor do bloco que a ventosa segura ou do bloco de cima da casa onde está: `"vermelho"`, `"azul"`, `"verde"` ou `"nenhuma"`.

NOTA

Os comandos escrevem-se sempre assim, sem acentos: `ir_para`, `avancar`. Se escreveres um que não existe, o simulador lista os que existem.

O que o Python do simulador aceita

LINGUAGEM

- Variáveis e contas: `+ - * // %`
- `if` / `elif` / `else`
- `for i in range(a, b)`
- `while condição:`
- `def nome(parametros):` (funções sem `return`)
- Comparações e `and` / `or` / `not`
- `pass`, `+=` e `-=`

LIMITES DE SEGURANÇA

- Máximo de **4000 passos** por execução (evita ciclos infinitos).
- Recursão até **40 níveis**.
- Sem `import`, ficheiros ou rede.
- Texto entre aspas simples ou duplas.
- Comentários com `#`.

Exemplo com ciclo e decisão

```python
# Separar 4 blocos do tapete por cor
for i in range(1, 5):
    tapete.avancar()
    braco.ir_para("T")
    braco.agarrar()
    if sensor.cor() == "vermelho":
        braco.ir_para("B4")
    else:
        braco.ir_para("B2")
    braco.largar()
```



Figura 6.1 · O editor Python com realce de linha.

Boas práticas

- Indenta com 4 espaços; é isso que define o corpo de um ciclo ou de um `if`.
- Dá nomes claros às tuas funções (`levar_bloco`, não `f1`).
- Se o simulador mostrar «linha 7», é mesmo a linha 7.
- Quando um programa fica longo, procura repetições para transformar em ciclos ou funções.

EXPERIMENTA

Corre a missão 2.1 em Python e depois muda `range(1, 5)` para `range(1, 4)`. Quantos blocos são levados? Porquê o último número **não** é incluído?

7 A mesa e as medidas

Geometria, alcance e coordenadas do robô

NESTE CAPÍTULO

- Ler a grelha A–E × 1–4 como o robô a lê
- Saber que casas são alcançáveis e porquê

A mesa tem **5 colunas (A a E)** e **4 linhas (1 a 4)**. Cada casa é um quadrado de **30 mm**. O Dobot fica na base, à esquerda do desenho, e todas as medidas estão em milímetros a partir do eixo do robô.

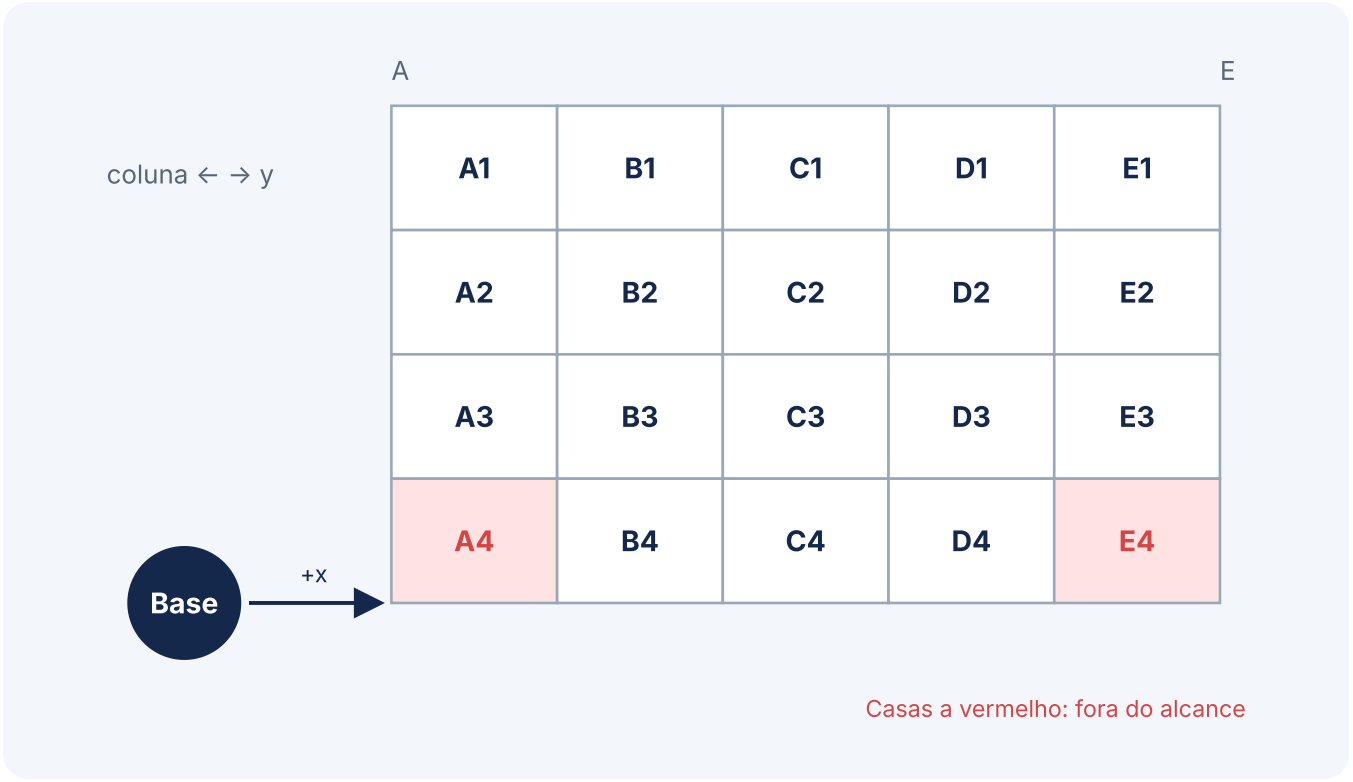


Figura 7.1 · Esquema simplificado da mesa, com a linha 4 mais perto da base. A1 e E1 estão a vermelho porque ficam fora do alcance do braço.

Coordenadas em milímetros

Casa	x (mm)	y (mm)
Linha 4	190	A = −60 · B = −30 · C = 0 · D = +30 · E = +60
Linha 3	220	
Linha 2	250	
Linha 1	280	

Em fórmula: `x = 190 + (4 − linha) × 30` e `y = (coluna − 3) × 30`, com a coluna a contar de 1 (A) a 5 (E). A **distância à base** é `√(x² + y²)` e aparece nos indicadores.

Alcance

Limite	Valor	Consequência
Alcance mínimo	165 mm	Mais perto da base o braço não consegue dobrar-se.
Alcance máximo	285 mm	Mais longe não consegue esticar-se.
Casas inalcançáveis	A1 e E1	Distância ≈ 286 mm. Mandar lá o braço dá erro de alcance.
Ponto T (tapete)	(230, 150)	Casa especial onde os blocos chegam ao fim do tapete.

> **ATENÇÃO**
> A missão 1.5 treina exatamente isto: encontrar um percurso que contorne as casas fora de alcance, em vez de ir diretamente a elas.

Cores dos blocos

Os blocos têm uma cor (vermelho, azul ou verde) e o sensor de cor lê a do bloco que está na casa onde o braço se encontra. Nas missões em que a cor interessa, a legenda na mesa e o enunciado indicam o que cada cor significa.

8 Tempo e eficiência

Quanto custa cada movimento

NESTE CAPÍTULO

- Prever quanto tempo demora um programa
- Distinguir instruções (estrelas) de tempo simulado

Cada comando consome tempo **simulado**, tal como no robô real. O tempo aparece no indicador do ecrã e é o mesmo a 1×, 2× ou 4×: a velocidade só acelera a animação.

Comando	Tempo simulado
`ir_para`	0,4 s + 0,25 s por cada 30 mm de distância + 0,08 s × altura (a partir do início, conta 2,5 casas)
`altura(n)`	0,3 s
`agarrar`	1,1 s
`largar`	1,0 s
`tapete.avancar`	0,8 s

Exemplo de cálculo

Ir de B2 a D2 (60 mm = 2 casas) com a altura por defeito (5): `0,4 + 0,25 × 2 + 0,08 × 5 = 1,3 s`. Um transporte completo (ir, agarrar, ir, largar) fica à volta de 6 segundos.

DICA

Estrelas e tempo são medidas diferentes. As estrelas contam **instruções** do teu programa; o tempo conta **o que o robô faz**. Um bom programa tenta ter poucas instruções e poucos movimentos inúteis. Percursos mais curtos poupam segundos.

O que acontece quando algo corre mal

Situação	Mensagem
Casa fora do alcance	«a casa A1 fica fora do alcance do braço (... mm da base)»
Casa que não existe	«a casa ... não existe (colunas A–E, linhas 1–4)»
Agarrar numa casa vazia	«não há nenhum bloco em ... para agarrar»
Largar sem bloco	«não tens nenhum bloco na ventosa»
Agarrar com a ventosa cheia	«a ventosa já tem um bloco ... Larga-o primeiro»
Torre acima de 3 blocos	«a torre em ... já tem 3 blocos»
Tapete bloqueado	«o tapete não pode avançar: ainda há um bloco por recolher no ponto T»
Ciclo infinito (mais de 4000 passos)	«o programa está a demorar demasiado (ciclo infinito?)»
Colisão ao atravessar uma torre	«colisão! ... Usa braco.altura(n) ou mais, ou contorna a torre» (capítulo 11)

9 Níveis e missões

O mapa completo das 26 missões

NESTE CAPÍTULO

- Ver o caminho de aprendizagem de ponta a ponta
- Saber que conceito trabalha cada missão

Cada nível acrescenta uma ideia à anterior. O «par» é o número de instruções para obter três estrelas na missão guiada.

Missão	Título	Conceito	Par	Linguagem
Nível 1 · Sequência — Mover blocos, uma instrução de cada vez				
1.1	Primeiro transporte	Sequência	4	Blocos ou Python
1.2	Dois blocos, dois destinos	Sequência	8	Blocos ou Python
1.3	Troca de lugares	Variável auxiliar	12	Blocos ou Python
1.4	Rotação de três	Variável auxiliar	16	Blocos ou Python
1.5	Junto ao limite	Alcance	12	Blocos ou Python
Nível 2 · Ciclos — Repetir em vez de copiar				
2.1	Repetir é poupar	Ciclo com contador	5	Blocos ou Python
2.2	Linha para linha	Ciclo com contador	5	Blocos ou Python
2.3	Escada	Ciclo com expressão	5	Blocos ou Python
2.4	Ciclos dentro de ciclos	Ciclos aninhados	6	Blocos ou Python
Nível 3 · Decisões — Usar o sensor de cor com se / senão				
3.1	Vermelho ou azul	Decisão	7	Blocos ou Python
3.2	Três cores	Decisão em cadeia	10	Blocos ou Python
3.3	Triagem da linha	Ciclo e decisão	6	Blocos ou Python
Nível 4 · Tapete — Blocos que chegam sozinhos num tapete rolante				
4.1	Ponto de recolha	Tapete rolante	4	Blocos ou Python
4.2	Cinco blocos	Ciclo com tapete	6	Blocos ou Python
4.3	Triagem no tapete	Tapete e decisão	9	Blocos ou Python
4.4	Até esvaziar	Enquanto e variáveis	9	Python
Nível 5 · Funções — Dar nome a um conjunto de instruções (só Python)				
5.1	Uma função para transportar	Funções	8	Python
5.2	Inverter a fila	Funções e variável auxiliar	15	Python
5.3	Uma função com ciclo	Função e ciclo	8	Python

Missão	Título	Conceito	Par	Linguagem
5.4	Triagem completa	Função, ciclo e decisão	14	Python
5.5	Três pilhas	Contadores e funções	20	Python
Nível 6 · Torres — Empilhar, desempilhar e evitar colisões				
6.1	Primeira torre	Empilhar	12	Blocos ou Python
6.2	Desmontar a torre	Agarrar o de cima	12	Blocos ou Python
6.3	Torre ao contrário	Ciclo e pilhas	5	Blocos ou Python
6.4	Evitar a colisão	Altura de viagem	5	Blocos ou Python
6.5	Duas torres	Triagem em torres	9	Blocos ou Python

As missões 4.4 e as do nível 5 só se resolvem em Python. Algumas missões têm vários cenários (por exemplo 3.2 tem 3) e outras têm cenários escondidos que só são revelados na validação.

Cenários escondidos

Algumas missões (3.3, 4.3, 4.4, 5.4, 5.5, 6.5) validam o programa também com situações que o aluno **não vê**, por exemplo outras cores ou outra ordem. Se a solução só funciona no exemplo (por exemplo, fixa cores em vez de ler o sensor), falha num cenário escondido e o simulador diz-lo. É a forma de garantir que o aluno escreveu um programa geral e não uma decoração do exemplo.

10 O tapete rolante

Receber blocos que chegam sozinhos

NESTE CAPÍTULO

- Usar `tapete.avancar()`, `tapete.tem_bloco()` e o ponto T
- Resolver o problema «quantos blocos vêm?» com ciclos

No nível 4 há um **tapete rolante** ao lado da mesa. Os blocos esperam numa fila e só chegam ao **ponto T** quando mandas o tapete avançar. O sensor de cor, no ponto T, diz a cor do bloco que chegou.

1. `tapete.avancar()` traz o próximo bloco para T (0,8 s).
2. `braco.ir_para("T")` posiciona a ventosa.
3. `braco.agarrar()` apanha o bloco.
4. Leva-o para o destino e `largar()`.



Figura 10.1 · Um bloco chegou ao ponto T.

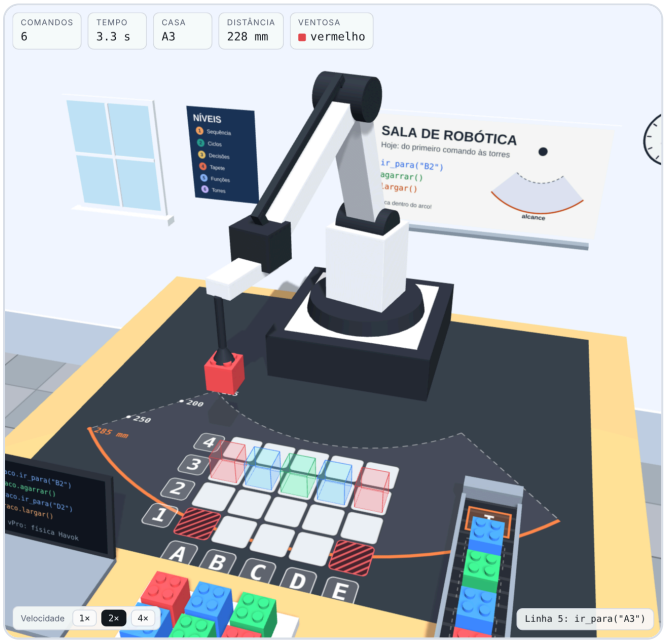


Figura 10.2 · No fim, todos os blocos foram arrumados.

Regras do tapete

- Só podes largar blocos nas casas da mesa, **nunca** no tapete.
- Se já há um bloco em T, o tapete não avança: tens de o recolher primeiro.
- Quando o tapete esvazia, `tapete.avancar()` não traz nenhum bloco e `tem_bloco()` é falso.
- Na missão 4.4 não sabes quantos blocos vêm: usa `while` com `tem_bloco()`.

Padrão: processar até acabar

```
# Padrão «até esvaziar» (missão 4.4)
tapete.avancar()
while tapete.tem_bloco():
    braco.ir_para("T")
    braco.agarrar()
    # ... decidir o destino pela cor ...
    braco.largar()
    tapete.avancar()
```

DICA

Repara que `avancar()` aparece **duas vezes**: uma antes do ciclo (para trazer o primeiro bloco) e outra no fim de cada volta (para trazer o seguinte).

EXPERIMENTA

Na missão 4.2, em vez de repetir 5 vezes, troca para `while tapete.tem_bloco():`. Funciona do mesmo modo? E se o tapete tivesse 7 blocos?

11 Torres e colisões

Empilhar em segurança (nível 6)

NESTE CAPÍTULO

- Construir e desmontar torres até 3 blocos
- Usar `braco.altura(n)` para evitar colisões

Na vPro, largar um bloco numa casa que já tem blocos põe-no **por cima**. Uma torre pode ter no máximo **3 blocos**. `agarrar()` tira sempre o bloco de cima. O bloco que chegou primeiro fica por baixo, por isso a ordem de empilhar decide a ordem da torre.

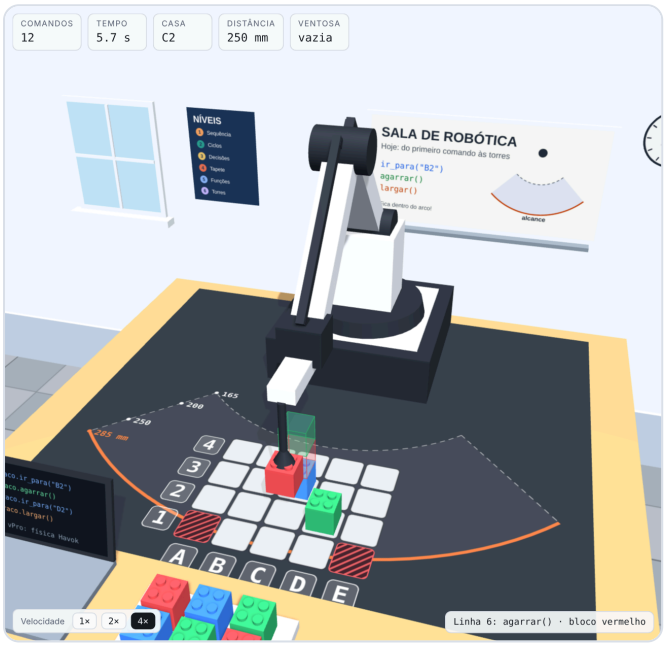


Figura 11.1 · Uma torre de três blocos em C3.

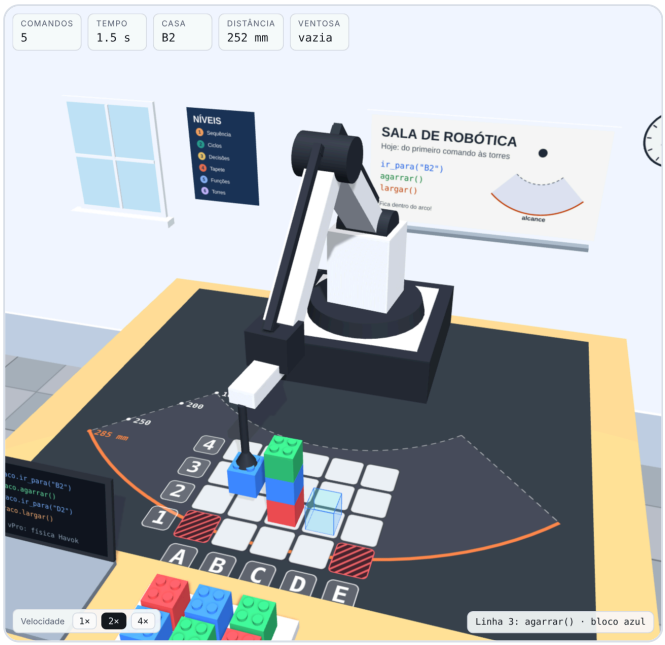


Figura 11.2 · A ventosa passa baixa e bate na torre.

A altura de viagem

A ventosa desloca-se **em linha reta** entre duas casas, a uma altura de 1 a 5 níveis de bloco. Por defeito é **5**, o que passa por cima de tudo. Baixar a altura poupa um pouco de tempo (0,08 s por nível), mas se uma torre estiver no caminho em linha reta, há colisão.

Situação	Altura mínima para passar
Ventosa vazia, torre de *k* blocos no caminho	**k**
A levar um bloco, torre de *k* blocos no caminho	**k + 1** (o bloco pendura-se um nível abaixo da ventosa)

Só contam as casas que a linha reta atravessa; a casa de onde se parte não conta. Se a altura é insuficiente, o simulador pára com a mensagem «colisão! … Usa braco.altura(n) ou mais, ou contorna a torre».

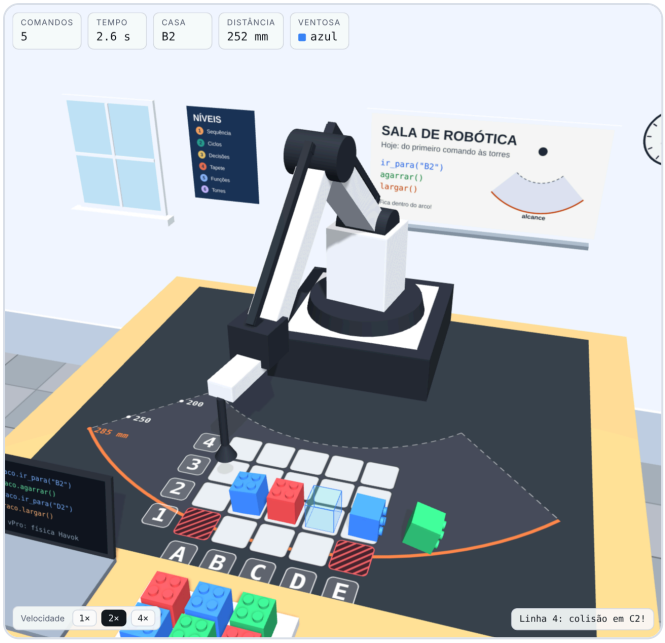


Figura 11.3 · O bloco levado derruba a torre.

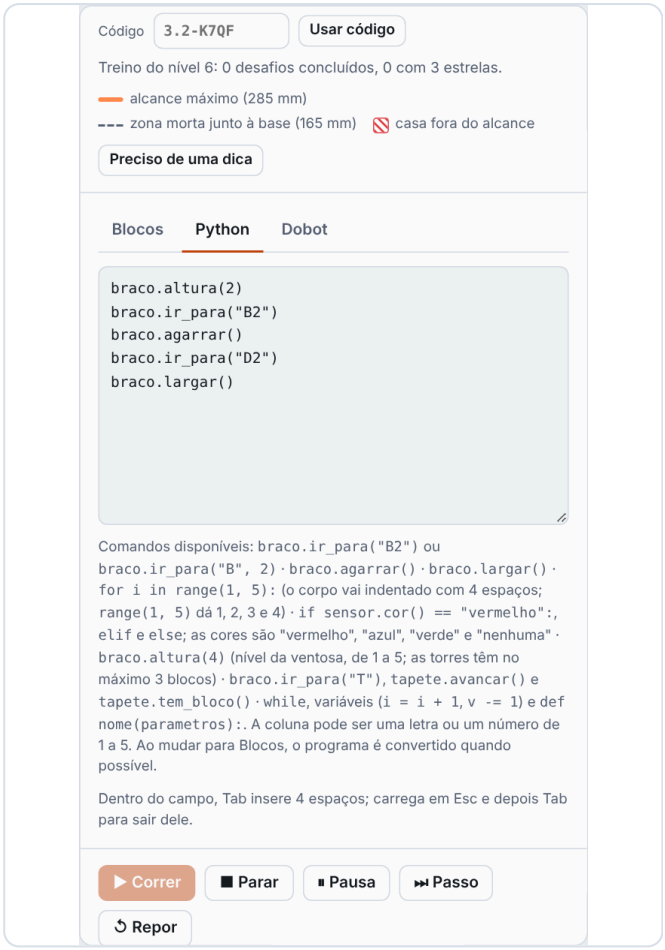


Figura 11.4 · O simulador explica o que correu mal.

> **ATENÇÃO**
>
> Com o motor físico, a torre cai mesmo (os blocos tombam). Sem WebAssembly, o resultado lógico é o mesmo mas a queda é uma animação simples.

Resolver uma colisão

1. Lê a mensagem: diz a casa da torre e quantos blocos tem.
2. Calcula a altura mínima pela tabela acima.
3. Coloca `braco.altura(n)` antes do `ir_para` que falhou, ou muda o percurso para contornar a torre.

```
braco.ir_para("B2")
braco.agarrar()
braco.altura(4)      # 3 blocos no caminho + 1 do bloco levado
braco.ir_para("D2")
braco.largar()
```

12 Desafios aleatórios

Treinar sem repetir sempre o mesmo exercício

NESTE CAPÍTULO

- Gerar desafios novos do mesmo tipo
- Partilhar um código com a turma

Cada uma das 26 missões tem um **gerador**: com ele o simulador cria muitas versões diferentes da mesma ideia (outras casas, cores, quantidades). Todas são verificadas para terem sempre solução e para a solução de referência cumprir o par.

Controlo	O que faz
Novo desafio	Cria uma variante da missão atual. Fica ativo depois de concluíres a missão guiada (ou com o modo professor).
Treino aleatório	Escolhe, ao acaso, um desafio de qualquer missão do mesmo nível, diferente do anterior.
Voltar à missão guiada	Regressa ao enunciado original.
Código	Todo o desafio tem um código, como `3.2-K7QF`. Quem escrever o mesmo código recebe o *mesmo* exercício.

Códigos para a turma

O código tem a forma `missão-XXXX`: o número da missão e quatro letras ou números. O professor escolhe um desafio, lê o código à turma e todos os alunos o escrevem no campo **Código** e carregam em **Usar código**. Assim todos resolvem o mesmo problema e podem comparar soluções.

DICA PARA O PROFESSOR

Para uma avaliação, dá a cada grupo um código diferente do mesmo tipo: o conceito é igual, mas não se pode copiar a resposta do colega.

Estrelas e treino

Num desafio aleatório as estrelas contam à parte, não substituem as da missão guiada. O painel mostra a contagem do treino do nível («Treino do nível 3: 5 desafios concluídos») e guarda-a no navegador do aluno.

EXPERIMENTA

Conclui a 2.1, carrega em **Novo desafio** e anota o código. Muda a página, escreve o código e verifica que o enunciado é o mesmo.

13 Do simulador ao Dobot real

Gerar o script e calibrar a mesa

NESTE CAPÍTULO

- Gerar o script para o Dobot Magician
- Calibrar a mesa real e testar em segurança

O separador **Dobot** traduz o teu programa para o Python do braço real (biblioteca `pydobot`). O script tem duas partes: um pequeno adaptador com os mesmos comandos do simulador (`braco.ir_para`, `braco.agarrar`, …) e, a seguir, o programa tal como o escreveste.

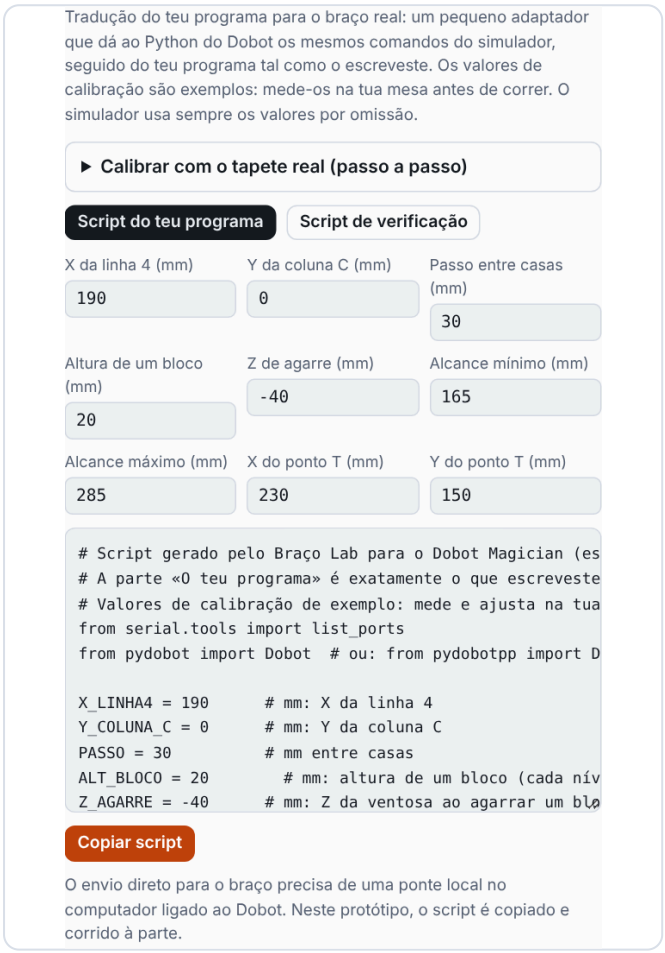


Figura 13.1 · O separador Dobot: calibração e script gerado.

Calibrar com a mesa real

Os valores são exemplos: mede-os na tua mesa.

1. Liga o braço ao DobotStudio e desliga a ventosa.
2. Move o braço à mão até a ponta ficar no centro da casa **C4** e lê X e Y. Repete para **C1**. (Opcional: **A4** para confirmar o passo.)
3. Com a ventosa encostada ao topo de um bloco, lê o **Z**. Mede também o **ponto T**.
4. Preenche os campos e carrega em **Aplicar medições**.
5. Carrega em **Script de verificação**: o braço visita os cantos (A4, E4, E2, A2, B1, D1, C3) sem agarrar nada. Corre-o a baixa velocidade.

Valores por omissão: X da linha 4 = 190 mm · Y da coluna C = 0 · passo = 30 mm · altura de um bloco = 20 mm · Z de agarre = −40 mm · alcance 165–285 mm · ponto T = (230, 150) mm.

SEGURANÇA

- No primeiro teste, usa velocidade baixa (o script já define 50) e a mão perto do botão de paragem.
- Confirma a porta série do braço antes de correr.
- O simulador usa sempre os valores por omissão; o que alteras na calibração só afeta o script gerado.
- Há avisos quando uma casa fica fora do arco com a tua calibração.

O envio direto para o braço precisa de uma ponte local no computador ligado ao Dobot: neste simulador o script é **copiado** com o botão **Copiar script** e corrido à parte, num ficheiro `.py`.

O botão copia o script para a área de transferência, pronto a colar num ficheiro `.py`.

14 Depurar e corrigir

Quando o programa não faz o que querias

NESTE CAPÍTULO

- Usar pausa e passo a passo
- Seguir um método para encontrar o erro

As ferramentas

- ⏸ **Pausa** e ⏭ **Passo** executam um comando de cada vez.
- A **linha atual** fica destacada no programa.
- As **variáveis** (`i`, contadores…) aparecem com o seu valor a cada passo.
- ↺ **Repor** devolve a mesa ao início do cenário.
- O **cenário** (botões 1, 2, 3) mostra a situação em que falhou.

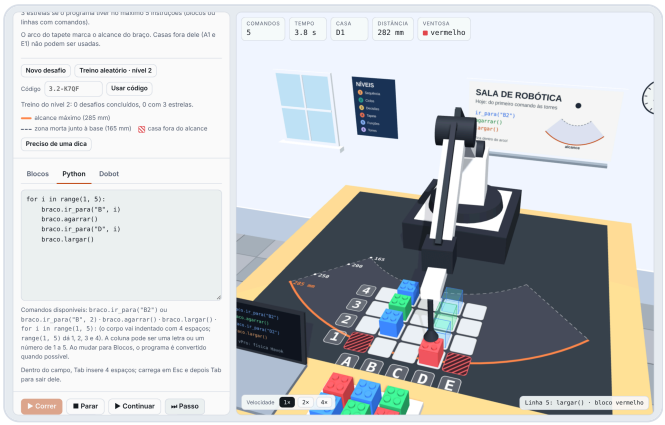


Figura 14.1 · Execução em pausa, com as variáveis do programa.

Um método em quatro passos

#	Passo
1	**Lê a mensagem inteira.** Diz a linha, a casa e o motivo.
2	**Reproduz o erro** no cenário certo (ou com ↺ Repor).
3	**Anda passo a passo** até ao ponto em que o robô faz algo que não esperavas e compara as variáveis com o que pensavas.
4	**Corrige uma coisa de cada vez** e volta a correr.

Erros frequentes

Sintoma	Causa provável
«fora do alcance»	Usaste A1 ou E1, ou uma casa fora do arco de 165–285 mm.
«não há nenhum bloco em …»	Foste à casa errada, ou a casa já foi esvaziada por um ciclo.
O último bloco não é movido	`range(1, 4)` acaba em 3: usa `range(1, 5)` para chegar a 4.
Funciona num cenário e falha noutro	Fixaste uma cor ou uma casa em vez de usar `sensor.cor()` ou a variável do ciclo.
«ciclo infinito?»	A condição do `while` nunca fica falsa (falta `tapete.avancar()` no fim).
Erro de indentação	Linhas dentro de `for`, `if` ou `def` têm de ter 4 espaços a mais.
«colisão!»	Passaste por cima de uma torre com altura insuficiente (capítulo 11).

15 Guia do professor

Preparar e conduzir uma aula

NESTE CAPÍTULO

- Planear uma aula de 50 minutos com o simulador
- Avaliar de forma justa e individual

Modo professor

A caixa **Modo professor** abre todos os níveis e liberta os desafios aleatórios em qualquer missão. Fica guardado no navegador. Desliga-o antes de entregar o computador aos alunos.

Uma aula tipo (50 min)

Tempo	Atividade
0–10	Demonstração da missão do dia no projetor. Pergunta: «O que vai fazer o braço?»
10–30	Trabalho em pares: um escreve, outro lê o enunciado e conta as casas. Trocam de papel a meio.
30–40	Desafio aleatório com um código dado pelo professor.
40–50	Fecho: duas soluções diferentes no quadro. Qual tem menos instruções? Qual é mais clara?

Competências trabalhadas

Nível	Pensamento computacional
1 Sequência	Decomposição e ordem das instruções; variável auxiliar.
2 Ciclos	Padrões e abstração; contadores e expressões.
3 Decisões	Condições e ramificação a partir de dados (cor).
4 Tapete	Processamento de dados em fluxo; ciclos `while`.
5 Funções	Modularização e reutilização de código.
6 Torres	Estruturas em pilha (último a entrar, primeiro a sair) e restrições físicas.

Avaliar

- Pede ao aluno para resolver um **desafio aleatório** com um código diferente do do colega.
- Valoriza a eficiência (estrelas) mas também a clareza: nomes bons e comentários.
- Usa os cenários escondidos como teste de generalização.
- Pede que expliquem o programa em voz alta, linha a linha.

SALA SEM COMPUTADORES PARA TODOS

Em tablets e telemóveis o painel passa para baixo da sala 3D, por isso funciona. Com um só computador, projeta o ecrã e deixa os alunos ditarem os comandos à vez.

16 Resolução de problemas

Quando é o simulador que não responde

Problema	O que fazer
Ecrã em branco ou sala preta	Confirma que o WebGL está ativo no navegador, atualiza-o e recarrega a página (Ctrl+F5).
A versão online não carrega	A rede da escola pode bloquear a rede de distribuição do Babylon.js e da física. Usa o ficheiro offline.
A animação é lenta	Fecha outros separadores, usa 2× ou 4× e, se for um computador antigo, a versão Three.js (mais leve).
Os blocos caem de forma estranha	Sem WebAssembly há uma queda simples. Em Chrome ou Edge recentes a física completa funciona.
As estrelas desapareceram	Foram apagados os dados do site ou mudou o navegador/computador. O progresso não é partilhado.
«Novo desafio» está desativado	Conclui a missão guiada, ou liga o modo professor.
Código «inválido»	Escreve-o como `3.2-K7QF` (missão, hífen, quatro caracteres).
Botão Blocos inativo	A missão é só em Python (4.4 e nível 5).
O script do Dobot falha à primeira	Verifica a porta série, a calibração (usa o Script de verificação) e a velocidade.
Não consigo copiar o script	Seleciona o texto na caixa e copia com Ctrl+C; alguns navegadores bloqueiam o botão.

EM CASO DE DÚVIDA

Recarrega a página: o simulador volta ao estado inicial e as estrelas guardadas mantêm-se. Se tiveres um programa importante, copia-o primeiro, porque o texto dos programas não fica guardado.

17 Ficha de consulta rápida

Tudo o que precisas numa página, e glossário

Comandos

```
braco.ir_para("B2")
braco.ir_para("T")
braco.agarrar()
braco.largar()
braco.altura(3)       # 1 a 5
tapete.avancar()
tapete.tem_bloco()
sensor.cor()        # vermelho | azul
                    # verde | nenhuma
```

Estruturas

```
for i in range(1, 5):
    ...
while condição:
    ...
if a == b: ...
elif a != c: ...
else: ...
def levar(a, b):
    ...
```

Números a decorar

Casa	30 mm
Mesa	A–E × 1–4
Alcance	165–285 mm
Fora de alcance	A1, E1
Ponto T	(230, 150)
Torre máx.	3 blocos
Altura	1 a 5 (def. 5)
Passos máx.	4000
agarrar / largar	1,1 s / 1,0 s
avançar tapete	0,8 s

Estrelas

★★★ ≤ par · ★★ ≤ 1,5 × par · ★ correto.

Glossário

Termo	Significado
Ventosa	Ponta do braço que agarra por sucção.
Par	Número de instruções que dá as três estrelas.
Ciclo	Conjunto de instruções repetido (`for`, `while`).
Condição	Pergunta com resposta sim/não que decide um caminho (`if`).
Função	Grupo de instruções com nome, definido com `def`.
Variável	Nome que guarda um valor (`i`, `n`).
Indentação	Recuo do texto que mostra o que pertence a um ciclo ou a um `if`.
Cenário	Uma situação inicial da mesa (blocos e cores) em que o programa é testado.
Colisão	Choque da ventosa ou do bloco com uma torre.
Calibração	Medida das posições reais para o script do Dobot.